SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment # 13

Name of Student : K.Saikiran
Enrollment No. : 2303A51539
Batch No. : 22

Task Description #1 (Refactoring – Removing Global Variables)
• Task: Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions.
o Refactor the code to pass values using function parameters.
o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1
def calculate_interest(amount):
return amount * rate
print(calculate_interest(1000))
• Expected Output:
o Refactored version passing rate as a parameter or using a configuration structure.

Prompt:-
write a python program on to eliminate unnecessary global variable and 0 identify global variable used across function and 0 refactor the code to pass values using function parameter.

Code:-

```
#Task-1:
#write a python program on to eliminate unneces
#sample code:



# Refactored code without global variable
def calculate_interest(amount, rate):

    return amount * rate
# Example usage
interest = calculate_interest(1000, 0.1)
print(interest)  # Output: 100.0
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON>
y
100.0
```

Justification:-
In the original code, the variable rate was declared globally and used inside the function. In the modified version, the rate is passed as a parameter to the function. This makes the function more flexible, reusable, and easier to maintain.

Task Description #2 : (Refactoring Deeply Nested Conditionals)
• Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas:
o Readability
o Logical simplification
o Maintainability
Legacy Code:

```
score = 78
if score >= 90:
print("Excellent")
else:
if score >= 75:
print("Very Good")
else:
if score >= 60:
print("Good")
else:
print("Needs Improvement")
```

Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

Prompt:-
write a python program on to refactor deeply if-elif-else logic into clear structure and 0 readability,0 logic simplification,0 maintainability.

Code:-

SR UNIVERSITY

```python
#Task-2:
#write a python program on to refact


# Refactored code with improved stru
def evaluate_score(score):

    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"
# Example usage
score = 78
result = evaluate_score(score)
print(result)
```

Output:-

```
PS C:\Users\Dell\Desktop\PYT
y
Very Good
```

Justification:-
The original program uses multiple nested conditions which makes the code harder to read. By simplifying the conditions into a flat structure, the code becomes easier to understand and maintain.

Task 3 (Refactoring Repeated File Handling Code)
• Task: Use AI to refactor repeated file open/read/close logic.
• Focus Areas:
o DRY principle
o Context managers
o Function reuse

Legacy Code:
f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()
Expected Outcome:
o Reusable function using with open() and parameters.

Prompt:-
Rewrite the code that repeatedly opens and reads files. Create a reusable function that reads the file using with open() so that the same function can be used for different files.

Code:-

```
# Task-3
# refactor the below code with repeated file h
f = open("C:\\AIAC LAB\\random.txt")
print(f.read())
f.close()
f = open("C:\\AIAC LAB\\random1.txt")
print(f.read())
f.close()
def read_file(file_name):
    with open(file_name, 'r') as f:
        return f.read()
print(read_file("C:\\AIAC LAB\\random.txt"))
print(read_file("C:\\AIAC LAB\\random1.txt"))
```

Output:-

```
This is a sample text file.
It contains plain text, no formatting.
1. Item one
2. Item two

This is the first text file.
It contains basic information.
Hello world.

This is a sample text file.
It contains plain text, no formatting.
1. Item one
2. Item two

This is the first text file.
It contains basic information.
Hello world.
```

Justification:-
The original code repeats the same steps for opening and reading files. The improved code creates a reusable function and uses a context manager to automatically close the file. This reduces repeated code and follows the DRY principle.

Task 4 (Optimizing Search Logic)
• Task: Refactor inefficient linear searches using appropriate data structures.
• Focus Areas:
o Time complexity
o Data structure choice

Legacy Code:
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Prompt:-
Refactor the given program that searches for a username in a list. Replace the list with a more efficient data structure such as a set to improve the search performance.

Code:-

```python
#Task-4:
#Refactor the given program that searches for a username in

def is_username_available(username, existing_usernames):
    """Check if the username is available."""
    return username not in existing_usernames
# Example usage
existing_usernames = {"user1", "user2", "user3"}
new_username = "user4"
if is_username_available(new_username, existing_usernames):
    print(f"Username '{new_username}' is available.")
else:
    print(f"Username '{new_username}' is already taken.")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON
y
Username 'user4' is available.
```

Justification:-
The original program searches through the list one by one which takes more time. Using a set allows faster lookup because it has constant time complexity for checking elements.

Task 5 (Refactoring Procedural Code into OOP Design)
• Task: Use AI to refactor procedural code into a class-based design.
• Focus Areas:
o Object-Oriented principles
o Encapsulation

Legacy Code:
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator with methods and attributes.

Prompt:-
Convert the given procedural code into an object-oriented program. Create a class that contains the salary information and a method to calculate the net salary.

Code:-

```
#Task-5:
#write a python program on Convert the given procedural code into a
class Employee:
    def __init__(self, name, gross_salary, tax_rate):
        self.name = name
        self.gross_salary = gross_salary
        self.tax_rate = tax_rate

    def calculate_net_salary(self):
        """Calculate and return the net salary after tax deduction.
        tax_amount = self.gross_salary * self.tax_rate
        net_salary = self.gross_salary - tax_amount
        return net_salary
# Example usage
employee = Employee("John Doe", 5000, 0.2)
net_salary = employee.calculate_net_salary()
print(f"Net Salary for {employee.name}: ${net_salary:.2f}")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> &
y
Net Salary for John Doe: $4000.00
```

Justification:-

The original code uses simple variables to calculate the salary. By converting it into a class-based structure, the data and related methods are grouped together which improves code organization and maintainability.

Task 6 (Refactoring for Performance Optimization)

• Task: Use AI to refactor a performance-heavy loop handling large data.

• Focus Areas:

o Algorithmic optimization

o Use of built-in functions

Legacy Code:

total = 0

for i in range(1, 1000000):

if i % 2 == 0:

total += i

print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Prompt:-
Refactor the given loop that calculates the sum of even numbers from a large range. Use Python built-in functions or mathematical logic to improve the performance of the program.

Code:-

```
#Task-6:
#write a python program on Refactor the given loop that calculates the sum o
def sum_of_even_numbers(n):
    """Calculate the sum of even numbers from 1 to n."""
    if n < 2:
        return 0
    # Using the formula for the sum of the first k even numbers: k * (k + 1)
    k = n // 2
    return k * (k + 1)
# Example usage
n = 1000000
result = sum_of_even_numbers(n)
print(f"The sum of even numbers from 1 to {n} is: {result}")
```

Output:-

```
PS C:\Users\Dell\Desktop\PYTHON> & C:/Users/Dell/AppData/Lo
y
The sum of even numbers from 1 to 1000000 is: 250000500000
```

Justification:-
The original program checks each number in a loop which takes more time for large values. The optimized solution uses a better approach to calculate the result faster and reduce unnecessary computation.

Task 7 (Removing Hidden Side Effects)
• Task: Refactor code that modifies shared mutable state.
• Focus Areas:
o Functional-style refactoring
o Predictability

Legacy Code:
data = []
def add_item(x):
data.append(x)
add_item(10)
add_item(20)

print(data)
Expected Outcome:
o Refactored function returning values instead of mutating globals.

Prompt:-
Rewrite the program so that the function does not directly modify the global list. Instead, return a new list with the added value.
Code:-

```
#Task-7:
#Write a python program on Rewrite the program so
def add_to_list(original_list, value):
    """Return a new list with the added value."""
    new_list = original_list.copy()
    new_list.append(value)
    return new_list
# Example usage
my_list = [1, 2, 3]
new_list = add_to_list(my_list, 4)
print("Original List:", my_list)
print("New List:", new_list)
```

Output:-

```
PS C:\Users\Dell\Desktop\
y
Original List: [1, 2, 3]
New List: [1, 2, 3, 4]
```

Justification:-
The original function modifies a shared list which can create side effects. The updated function returns a new list instead of modifying the existing one, making the program safer and easier to understand.

Task 8 (Refactoring Complex Input Validation Logic)
• Task: Use AI to simplify and modularize complex validation rules.
• Focus Areas: Readability, Testability

Legacy Code:
password = input("Enter password: ")
if len(password) >= 8:
if any(c.isdigit() for c in password):
if any(c.isupper() for c in password):
print("Valid Password")

else:
print("Must contain uppercase")
else:
print("Must contain digit")
else:
print("Password too short")
Expected Outcome:
o Separate validation functions with clear responsibility

Prompt:-
Improve the password validation code by simplifying the nested conditions. Use built-in Python functions to check the required conditions such as length, digits, and uppercase letters.
Code:-

```
#Task-8:
#write a python program on Improve the password validation code by sim
def validate_password(password):
    """Validate the password based on given criteria."""
    if len(password) < 8:
        return "Password must be at least 8 characters long."
    if not any(char.isdigit() for char in password):
        return "Password must contain at least one digit."
    if not any(char.isupper() for char in password):
        return "Password must contain at least one uppercase letter."
    return "Password is valid."
# Example usage
password = "Password123"
validation_result = validate_password(password)
print(validation_result)
```

Output:-

```
PS C:\Users\Dell\Desktop\PY
y
Password is valid.
```

Justification:-
The original program uses multiple nested conditions to validate the password. The improved code simplifies the logic using built-in functions which makes the program shorter, clearer, and easier to test.